Module 6: Exception Handling

Experiment 1: Division by Zero Exception

Aim

To handle the `ZERO_DIVIDE` exception.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    num1 NUMBER := 100;
    num2 NUMBER := 0;
    result NUMBER;
BEGIN
    result := num1 / num2;
    DBMS_OUTPUT.PUT_LINE('Result = ' || result);

EXCEPTION
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('Error: Division by zero is not allowed.');
END;
/
```

Output

```
Error: Division by zero is not allowed.
```

Experiment 2: Invalid Number (VALUE_ERROR)

Aim

To handle the `VALUE_ERROR` exception.

Program

```
SET SERVEROUTPUT ON;

DECLARE
    age NUMBER(2);
BEGIN
```

```
    age := 150;

    DBMS_OUTPUT.PUT_LINE(age);

EXCEPTION
    WHEN VALUE_ERROR THEN
        DBMS_OUTPUT.PUT_LINE('Value Error: Number exceeds the defined
precision.');
END;
/
```

Output

```
Value Error: Number exceeds the defined precision.
```

Experiment 3: NO_DATA_FOUND Exception

Aim

To demonstrate the `NO_DATA_FOUND` exception.

Required Table

```
CREATE TABLE employee
(
    emp_id NUMBER PRIMARY KEY,
    emp_name VARCHAR2(30),
    salary NUMBER
);

INSERT INTO employee VALUES(101,'Rahul',45000);
INSERT INTO employee VALUES(102,'Amit',50000);

COMMIT;
```

Program

```
SET SERVEROUTPUT ON;

DECLARE
    v_name employee.emp_name%TYPE;
BEGIN

    SELECT emp_name
    INTO v_name
    FROM employee
    WHERE emp_id = 999;
```

```
    DBMS_OUTPUT.PUT_LINE(v_name);

EXCEPTION

    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee Record Not Found.');

END;
/
```

Output

```
Employee Record Not Found.
```

Experiment 4: TOO_MANY_ROWS Exception

Aim

To demonstrate the `TOO_MANY_ROWS` exception.

Program

```
SET SERVEROUTPUT ON;

DECLARE

    v_name employee.emp_name%TYPE;

BEGIN

    SELECT emp_name
    INTO v_name
    FROM employee
    WHERE salary > 40000;

    DBMS_OUTPUT.PUT_LINE(v_name);

EXCEPTION

    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('More than one employee found.');

END;
/
```

Output

```
More than one employee found.
```

Experiment 5: User Defined Exception

Aim

To create and handle a user-defined exception.

Program

```
SET SERVEROUTPUT ON;

DECLARE

    marks NUMBER := 35;

    student_failed EXCEPTION;

BEGIN

    IF marks < 40 THEN
        RAISE student_failed;
    END IF;

    DBMS_OUTPUT.PUT_LINE('Student Passed');

EXCEPTION

    WHEN student_failed THEN
        DBMS_OUTPUT.PUT_LINE('Student Failed.');

END;
/
```

Output

```
Student Failed.
```

Experiment 6: ATM Balance Exception

Aim

To demonstrate a user-defined exception for insufficient account balance.

Program

```
SET SERVEROUTPUT ON;

DECLARE

    balance NUMBER := 3000;
    withdraw_amount NUMBER := 5000;

    insufficient_balance EXCEPTION;

BEGIN

    IF withdraw_amount > balance THEN
        RAISE insufficient_balance;
    END IF;

    balance := balance - withdraw_amount;

    DBMS_OUTPUT.PUT_LINE('Transaction Successful');
    DBMS_OUTPUT.PUT_LINE('Remaining Balance = ' || balance);

EXCEPTION

    WHEN insufficient_balance THEN
        DBMS_OUTPUT.PUT_LINE('Transaction Failed: Insufficient Balance.');

END;
/
```

Output

```
Transaction Failed: Insufficient Balance.
```

Experiment 7: Banking Withdrawal Validation

Aim

To validate withdrawal while maintaining the minimum account balance.

Program

```
SET SERVEROUTPUT ON;

DECLARE

    balance NUMBER := 10000;
    withdrawal NUMBER := 9500;
    minimum_balance CONSTANT NUMBER := 1000;
```

```
BEGIN

    IF balance - withdrawal < minimum_balance THEN

        RAISE_APPLICATION_ERROR(
            -20001,
            'Minimum balance of Rs.1000 should be maintained.'
        );

    ELSE

        balance := balance - withdrawal;

        DBMS_OUTPUT.PUT_LINE('Transaction Successful');
        DBMS_OUTPUT.PUT_LINE('Remaining Balance = ' || balance);

    END IF;

END;
/
```

Output

```
ORA-20001:
Minimum balance of Rs.1000 should be maintained.
```

Experiment 8: Employee Salary Validation

Aim

To validate the minimum employee salary.

Program

```
SET SERVEROUTPUT ON;

DECLARE

    salary NUMBER := 12000;

BEGIN

    IF salary < 15000 THEN

        RAISE_APPLICATION_ERROR(
            -20002,
            'Employee salary cannot be less than Rs.15000.'
        );

    ELSE
```

```
        DBMS_OUTPUT.PUT_LINE('Salary Validation Successful.');

    END IF;

END;
/
```

Output

```
ORA-20002:
Employee salary cannot be less than Rs.15000.
```

Experiment 9: Student Admission Validation

Aim

To validate student admission based on minimum qualifying marks.

Program

```
SET SERVEROUTPUT ON;

DECLARE

    percentage NUMBER := 45;

BEGIN

    IF percentage < 50 THEN

        RAISE_APPLICATION_ERROR(
            -20003,
            'Admission Rejected: Minimum 50% marks required.'
        );

    ELSE

        DBMS_OUTPUT.PUT_LINE('Admission Granted.');

    END IF;

END;
/
```

Output

```
ORA-20003:
Admission Rejected: Minimum 50% marks required.
```

Experiment 10: Library Book Issue Validation

Aim

To check the availability of a book before issuing it.

Program

```
SET SERVEROUTPUT ON;

DECLARE

    book_status VARCHAR2(20) := 'Issued';

BEGIN

    IF UPPER(book_status) = 'ISSUED' THEN

        RAISE_APPLICATION_ERROR(
            -20004,
            'Book is already issued to another student.'
        );

    ELSE

        DBMS_OUTPUT.PUT_LINE('Book Issued Successfully.');

    END IF;

END;
/
```

Output

```
ORA-20004:
Book is already issued to another student.
```

Experiment 11: Using SQLCODE and SQLERRM

Aim

To display Oracle error code and error message.

Program

```
SET SERVEROUTPUT ON;

DECLARE

    a NUMBER := 100;
    b NUMBER := 0;
    c NUMBER;

BEGIN

    c := a / b;

EXCEPTION

    WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE('Error Code    : ' || SQLCODE);
        DBMS_OUTPUT.PUT_LINE('Error Message : ' || SQLERRM);

END;
/
```

Output

```
Error Code    : -1476
Error Message : ORA-01476: divisor is equal to zero
```

Experiment 12: Nested Exception Handling

Aim

To demonstrate nested exception handling in PL/SQL.

Program

```
SET SERVEROUTPUT ON;

BEGIN

    BEGIN

        DECLARE
            a NUMBER := 10;
            b NUMBER := 0;
            c NUMBER;
        BEGIN
            c := a / b;
        END;
```

```
    EXCEPTION
        WHEN ZERO_DIVIDE THEN
            DBMS_OUTPUT.PUT_LINE('Inner Block: Division by Zero Handled.');
    END;

    DBMS_OUTPUT.PUT_LINE('Outer Block Executed Successfully.');

END;
/
```

Output

```
Inner Block: Division by Zero Handled.
Outer Block Executed Successfully.
```

Additional Practice Programs

Students can further practice exception handling with the following exercises:

1. Electricity Bill Validation
2. Credit Card Transaction Limit Validation
3. Online Shopping Stock Validation
4. Railway Seat Availability Check
5. Movie Ticket Booking Validation
6. Hospital Appointment Scheduling
7. Driving License Age Validation
8. Passport Eligibility Validation
9. Insurance Premium Validation
10. Hostel Room Allocation Validation
11. Employee Leave Approval Validation
12. Scholarship Eligibility Validation
13. Online Examination Login Validation
14. Mobile Banking Transaction Limit Check
15. Loan Approval Eligibility Validation

Module 7: Cursors in PL/SQL

Database Table Required

Before performing the experiments, create the **EMPLOYEE** table.

Employee Table

```
CREATE TABLE employee
(
    emp_id NUMBER PRIMARY KEY,
    emp_name VARCHAR2(30),
    department VARCHAR2(30),
    salary NUMBER
);
```

Insert Sample Records

```
INSERT INTO employee VALUES(101,'Rahul','IT',45000);
INSERT INTO employee VALUES(102,'Amit','HR',50000);
INSERT INTO employee VALUES(103,'Priya','IT',60000);
INSERT INTO employee VALUES(104,'Neha','Finance',55000);
INSERT INTO employee VALUES(105,'Ankit','IT',48000);

COMMIT;
```

Experiment 1: Display Employee Records

Aim

To display all employee records using an explicit cursor.

Program

```
SET SERVEROUTPUT ON;

DECLARE

CURSOR emp_cursor IS

SELECT *

FROM employee;

v_id employee.emp_id%TYPE;
```

```
v_name employee.emp_name%TYPE;
v_dept employee.department%TYPE;
v_salary employee.salary%TYPE;

BEGIN

OPEN emp_cursor;

LOOP

FETCH emp_cursor
INTO v_id,v_name,v_dept,v_salary;

EXIT WHEN emp_cursor%NOTFOUND;

DBMS_OUTPUT.PUT_LINE
(
v_id||' '||
v_name||' '||
v_dept||' '||
v_salary
);

END LOOP;

CLOSE emp_cursor;

END;
/
```

Output

```
101 Rahul IT 45000
102 Amit HR 50000
103 Priya IT 60000
104 Neha Finance 55000
105 Ankit IT 48000
```

Experiment 2: Display Employee Salary

Aim

To display employee names and salaries.

Program

```
SET SERVEROUTPUT ON;

DECLARE

CURSOR salary_cursor IS
```

```
SELECT emp_name,salary
FROM employee;

v_name employee.emp_name%TYPE;
v_salary employee.salary%TYPE;

BEGIN

OPEN salary_cursor;

LOOP

FETCH salary_cursor
INTO v_name,v_salary;

EXIT WHEN salary_cursor%NOTFOUND;

DBMS_OUTPUT.PUT_LINE
(
v_name||' Salary = '||v_salary
);

END LOOP;

CLOSE salary_cursor;

END;
/
```

Output

```
Rahul Salary = 45000
Amit Salary = 50000
Priya Salary = 60000
Neha Salary = 55000
Ankit Salary = 48000
```

Experiment 3: Department-wise Employees

Aim

To display employees belonging to the IT department.

Program

```
SET SERVEROUTPUT ON;

DECLARE
```

```
CURSOR dept_cursor IS

SELECT emp_name,salary

FROM employee

WHERE department='IT';

v_name employee.emp_name%TYPE;
v_salary employee.salary%TYPE;

BEGIN

OPEN dept_cursor;

LOOP

FETCH dept_cursor
INTO v_name,v_salary;

EXIT WHEN dept_cursor%NOTFOUND;

DBMS_OUTPUT.PUT_LINE
(
v_name||'  '||v_salary
);

END LOOP;

CLOSE dept_cursor;

END;
/
```

Output

```
Rahul 45000
Priya 60000
Ankit 48000
```

Experiment 4: Parameterized Cursor

Aim

To demonstrate a parameterized cursor.

Program

```
SET SERVEROUTPUT ON;
```

```
DECLARE

CURSOR dept_cursor
(
dept_name VARCHAR2
)

IS

SELECT emp_name,salary

FROM employee

WHERE department=dept_name;

v_name employee.emp_name%TYPE;
v_salary employee.salary%TYPE;

BEGIN

OPEN dept_cursor('Finance');

LOOP

FETCH dept_cursor
INTO v_name,v_salary;

EXIT WHEN dept_cursor%NOTFOUND;

DBMS_OUTPUT.PUT_LINE
(
v_name||'  '||v_salary
);

END LOOP;

CLOSE dept_cursor;

END;
/
```

Output

```
Neha 55000
```

Experiment 5: Cursor FOR LOOP

Aim

To display employee records using Cursor FOR LOOP.

Program

```
SET SERVEROUTPUT ON;

DECLARE

CURSOR emp_cursor IS

SELECT emp_name,salary

FROM employee;

BEGIN

FOR emp_record IN emp_cursor LOOP

DBMS_OUTPUT.PUT_LINE
(
emp_record.emp_name||
' Salary = '||
emp_record.salary
);

END LOOP;

END;
/
```

Output

```
Rahul Salary = 45000
Amit Salary = 50000
Priya Salary = 60000
Neha Salary = 55000
Ankit Salary = 48000
```

Experiment 6: Calculate Total Salary

Aim

To calculate the total salary using a cursor.

Program

```
SET SERVEROUTPUT ON;

DECLARE

CURSOR emp_cursor IS
```

```
SELECT salary

FROM employee;

v_salary NUMBER;

total_salary NUMBER:=0;

BEGIN

OPEN emp_cursor;

LOOP

FETCH emp_cursor
INTO v_salary;

EXIT WHEN emp_cursor%NOTFOUND;

total_salary:=total_salary+v_salary;

END LOOP;

CLOSE emp_cursor;

DBMS_OUTPUT.PUT_LINE
(
'Total Salary = '||
total_salary
);

END;
/
```

Output

```
Total Salary = 258000
```

Experiment 7: Calculate Average Salary

Aim

To calculate the average salary using a cursor.

Program

```
SET SERVEROUTPUT ON;

DECLARE

CURSOR emp_cursor IS
```

```
SELECT salary

FROM employee;

v_salary NUMBER;

total NUMBER:=0;

count_emp NUMBER:=0;

average_salary NUMBER;

BEGIN

OPEN emp_cursor;

LOOP

FETCH emp_cursor INTO v_salary;

EXIT WHEN emp_cursor%NOTFOUND;

total:=total+v_salary;

count_emp:=count_emp+1;

END LOOP;

CLOSE emp_cursor;

average_salary:=total/count_emp;

DBMS_OUTPUT.PUT_LINE
(
'Average Salary = '||
ROUND(average_salary,2)
);

END;
/
```

Output

```
Average Salary = 51600
```

Experiment 8: Employee Count

Aim

To count the total number of employees using a cursor.

Program

```
SET SERVEROUTPUT ON;

DECLARE

CURSOR emp_cursor IS

SELECT emp_name

FROM employee;

v_name employee.emp_name%TYPE;

count_emp NUMBER:=0;

BEGIN

OPEN emp_cursor;

LOOP

FETCH emp_cursor
INTO v_name;

EXIT WHEN emp_cursor%NOTFOUND;

count_emp:=count_emp+1;

END LOOP;

CLOSE emp_cursor;

DBMS_OUTPUT.PUT_LINE
(
'Total Employees = '||
count_emp
);

END;
/
```

Output

```
Total Employees = 5
```

Experiment 9: Salary Increment

Aim

To increase employee salary by 10% using a cursor.

Program

```
SET SERVEROUTPUT ON;

DECLARE

CURSOR emp_cursor IS

SELECT emp_id,salary

FROM employee

FOR UPDATE;

v_id employee.emp_id%TYPE;
v_salary employee.salary%TYPE;

BEGIN

OPEN emp_cursor;

LOOP

FETCH emp_cursor
INTO v_id,v_salary;

EXIT WHEN emp_cursor%NOTFOUND;

UPDATE employee

SET salary=salary*1.10

WHERE CURRENT OF emp_cursor;

END LOOP;

CLOSE emp_cursor;

COMMIT;

DBMS_OUTPUT.PUT_LINE('Salary Updated Successfully.');

END;
/
```

Output

```
Salary Updated Successfully.
```

Experiment 10: Student Result Cursor

Aim

To display student details using a cursor.

Create Table

```
CREATE TABLE student
(
roll_no NUMBER,
student_name VARCHAR2(30),
marks NUMBER
);
```

Insert Records

```
INSERT INTO student VALUES(1,'Rahul',85);
INSERT INTO student VALUES(2,'Amit',72);
INSERT INTO student VALUES(3,'Neha',91);

COMMIT;
```

Program

```
SET SERVEROUTPUT ON;

DECLARE

CURSOR student_cursor IS

SELECT student_name,marks

FROM student;

v_name student.student_name%TYPE;
v_marks student.marks%TYPE;

BEGIN

OPEN student_cursor;

LOOP

FETCH student_cursor
INTO v_name,v_marks;

EXIT WHEN student_cursor%NOTFOUND;

DBMS_OUTPUT.PUT_LINE
(
v_name||
' Marks = '||
v_marks
);
```

```
END LOOP;

CLOSE student_cursor;

END;
/
```

Output

```
Rahul Marks = 85
Amit Marks = 72
Neha Marks = 91
```

Bonus Experiment 11: Cursor Attributes

Aim

To demonstrate cursor attributes.

Program

```
SET SERVEROUTPUT ON;

DECLARE

CURSOR emp_cursor IS
SELECT emp_name FROM employee;

v_name employee.emp_name%TYPE;

BEGIN

OPEN emp_cursor;

LOOP

FETCH emp_cursor INTO v_name;

EXIT WHEN emp_cursor%NOTFOUND;

DBMS_OUTPUT.PUT_LINE(
'Record ' ||
emp_cursor%ROWCOUNT ||
' : ' ||
v_name);

END LOOP;

CLOSE emp_cursor;
```

```
END;
/
```

Bonus Experiment 12: Cursor FOR LOOP with Parameters

```
SET SERVEROUTPUT ON;

DECLARE

CURSOR dept_cursor(p_dept VARCHAR2)
IS
SELECT emp_name,salary
FROM employee
WHERE department=p_dept;

BEGIN

FOR emp_rec IN dept_cursor('IT') LOOP

DBMS_OUTPUT.PUT_LINE(
emp_rec.emp_name||
' '||
emp_rec.salary);

END LOOP;

END;
/
```

Additional Practice Programs

Students can further strengthen their understanding of cursors through the following exercises:

1. Display employees earning more than ₹50,000 using an explicit cursor.
2. Display employees department-wise using a parameterized cursor.
3. Count the number of employees in each department.
4. Display the highest-paid employee using a cursor.
5. Display the lowest-paid employee using a cursor.
6. Generate a salary report with employee names and annual salaries.
7. Update salaries of employees in a selected department using `WHERE CURRENT OF`.
8. Display employees whose names start with the letter 'A'.
9. Generate employee experience reports using a cursor.
10. Calculate department-wise total salaries using parameterized cursors.
11. Create a cursor to generate student grades based on marks.
12. Display employees sorted by salary in descending order using a cursor.

13. Demonstrate the use of `%FOUND`, `%NOTFOUND`, `%ROWCOUNT`, and `%ISOPEN`.
14. Compare explicit cursors with Cursor FOR LOOP implementations.
15. Create a menu-driven employee management program using cursors.

These experiments provide students with comprehensive practical exposure to **Explicit Cursors**, **Parameterized Cursors**, **Cursor FOR Loops**, **Cursor Attributes**, and **cursor-based data processing**, preparing them for university practical examinations and enterprise database programming.

Module 8: Procedures and Functions

Database Table Required

Employee Table

```
CREATE TABLE employee
(
    emp_id NUMBER PRIMARY KEY,
    emp_name VARCHAR2(30),
    department VARCHAR2(20),
    salary NUMBER
);

INSERT INTO employee VALUES (101,'Rahul','IT',45000);
INSERT INTO employee VALUES (102,'Amit','HR',50000);
INSERT INTO employee VALUES (103,'Priya','Finance',60000);

COMMIT;
```

Experiment 1: Simple Procedure

Aim

To create and execute a simple procedure.

Program

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE display_message
IS
BEGIN
    DBMS_OUTPUT.PUT_LINE('Welcome to PL/SQL Programming');
END;
/

BEGIN
    display_message;
END;
/
```

Output

```
Welcome to PL/SQL Programming
```

Experiment 2: Procedure with IN Parameter

Aim

To demonstrate a procedure using an IN parameter.

Program

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE square_number
(
    num IN NUMBER
)
IS
BEGIN
    DBMS_OUTPUT.PUT_LINE('Square = ' || (num*num));
END;
/

BEGIN
    square_number(12);
END;
/
```

Output

```
Square = 144
```

Experiment 3: Procedure with OUT Parameter

Aim

To demonstrate a procedure using an OUT parameter.

Program

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE addition
(
    a IN NUMBER,
```

```
    b IN NUMBER,
    result OUT NUMBER
)
IS
BEGIN
    result := a + b;
END;
/

DECLARE
    total NUMBER;
BEGIN
    addition(20,30,total);

    DBMS_OUTPUT.PUT_LINE('Sum = ' || total);
END;
/
```

Output

```
Sum = 50
```

Experiment 4: Procedure with IN OUT Parameter

Aim

To demonstrate a procedure using an IN OUT parameter.

Program

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE increment_salary
(
    salary IN OUT NUMBER
)
IS
BEGIN
    salary := salary + 5000;
END;
/

DECLARE
    emp_salary NUMBER := 40000;
BEGIN

    increment_salary(emp_salary);
```

```
    DBMS_OUTPUT.PUT_LINE('Updated Salary = ' || emp_salary);

END;
/
```

Output

```
Updated Salary = 45000
```

Experiment 5: Addition Function

Aim

To create a function that returns the sum of two numbers.

Program

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION add_numbers
(
    a NUMBER,
    b NUMBER
)

RETURN NUMBER

IS

BEGIN

    RETURN a+b;

END;
/

BEGIN

DBMS_OUTPUT.PUT_LINE
(
'Sum = '||
add_numbers(25,15)
);

END;
/
```

Output

```
Sum = 40
```

Experiment 6: Factorial Function

Aim

To calculate the factorial of a number using a function.

Program

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION factorial
(
    n NUMBER
)

RETURN NUMBER

IS

fact NUMBER:=1;

BEGIN

FOR i IN 1..n LOOP

fact:=fact*i;

END LOOP;

RETURN fact;

END;
/

BEGIN

DBMS_OUTPUT.PUT_LINE
(
'Factorial = '||
factorial(5)
);

END;
/
```

Output

```
Factorial = 120
```

Experiment 7: Prime Number Function

Aim

To determine whether a number is prime using a function.

Program

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION is_prime
(
    n NUMBER
)

RETURN VARCHAR2

IS

count_factor NUMBER:=0;

BEGIN

FOR i IN 1..n LOOP

IF MOD(n,i)=0 THEN
count_factor:=count_factor+1;
END IF;

END LOOP;

IF count_factor=2 THEN
RETURN 'Prime';
ELSE
RETURN 'Not Prime';
END IF;

END;
/

BEGIN

DBMS_OUTPUT.PUT_LINE
(
is_prime(17)
);

END;
/
```

Output

```
Prime
```

Experiment 8: GST Calculator Function

Aim

To calculate GST using a function.

Program

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION calculate_gst
(
    amount NUMBER
)

RETURN NUMBER

IS

BEGIN

RETURN amount*0.18;

END;
/

BEGIN

DBMS_OUTPUT.PUT_LINE
(
'GST = '||
calculate_gst(25000)
);

END;
/
```

Output

```
GST = 4500
```

Experiment 9: Largest Number Function

Aim

To find the largest of two numbers using a function.

Program

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION largest
(
    a NUMBER,
    b NUMBER
)

RETURN NUMBER

IS

BEGIN

IF a>b THEN
RETURN a;
ELSE
RETURN b;
END IF;

END;
/

BEGIN

DBMS_OUTPUT.PUT_LINE
(
'Largest Number = '||
largest(45,90)
);

END;
/
```

Output

```
Largest Number = 90
```

Experiment 10: Employee Salary Update Procedure

Aim

To update an employee's salary using a stored procedure.

Program

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE update_salary
(
    p_emp_id IN NUMBER,
    p_increment IN NUMBER
)

IS

BEGIN

UPDATE employee

SET salary=salary+p_increment

WHERE emp_id=p_emp_id;

COMMIT;

DBMS_OUTPUT.PUT_LINE
(
'Salary Updated Successfully.'
);

END;
/

BEGIN

update_salary(101,5000);

END;
/
```

Verification

```
SELECT * FROM employee WHERE emp_id=101;
```

Output

```
EMP_ID   EMP_NAME   DEPARTMENT   SALARY
--------------------------------------
101      Rahul      IT           50000
```

Bonus Experiment 11: Function to Calculate Annual Salary

Aim

To calculate an employee's annual salary.

Program

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION annual_salary
(
    monthly_salary NUMBER
)

RETURN NUMBER

IS

BEGIN

RETURN monthly_salary*12;

END;
/

BEGIN

DBMS_OUTPUT.PUT_LINE
(
'Annual Salary = '||
annual_salary(50000)
);

END;
/
```

Output

```
Annual Salary = 600000
```

Bonus Experiment 12: Procedure to Display Employee Details

Aim

To display employee details using a stored procedure.

Program

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE employee_details
(
    p_emp_id NUMBER
)

IS

v_name employee.emp_name%TYPE;
v_dept employee.department%TYPE;
v_salary employee.salary%TYPE;

BEGIN

SELECT emp_name,department,salary

INTO v_name,v_dept,v_salary

FROM employee

WHERE emp_id=p_emp_id;

DBMS_OUTPUT.PUT_LINE('Employee Name : '||v_name);
DBMS_OUTPUT.PUT_LINE('Department    : '||v_dept);
DBMS_OUTPUT.PUT_LINE('Salary        : '||v_salary);

END;
/

BEGIN

employee_details(102);

END;
/
```

Output

```
Employee Name : Amit
Department    : HR
Salary        : 50000
```

Additional Practice Programs

Students can further strengthen their understanding by implementing the following:

1. Procedure to calculate the area of a circle.
2. Procedure to generate the multiplication table of a number.
3. Function to calculate compound interest.
4. Function to determine whether a number is even or odd.
5. Function to reverse a number.
6. Procedure to calculate a student's grade.
7. Function to calculate income tax.
8. Procedure to transfer money between two bank accounts.
9. Procedure to update employee department details.
10. Function to determine the highest salary among employees.
11. Procedure to display all employee records.
12. Function to calculate the average marks of students.
13. Procedure to apply a percentage-based salary increment.
14. Function to count the number of employees in a department.
15. Procedure to generate a payroll report.

Viva Questions

1. What is a stored procedure in PL/SQL?
2. What is a function? How does it differ from a procedure?
3. Explain the differences between `IN`, `OUT`, and `IN OUT` parameters.
4. Can a function return multiple values? Why or why not?
5. When should you use a procedure instead of a function?
6. What are the advantages of stored procedures?
7. What are the advantages of user-defined functions?
8. Can a procedure call another procedure or function?
9. How are procedures and functions stored in the Oracle database?
10. What are the real-world applications of procedures and functions in enterprise systems?

These experiments provide students with practical exposure to **stored procedures**, **user-defined functions**, **parameter passing mechanisms**, and **database updates**, which are essential concepts for Oracle PL/SQL development and enterprise database programming.